

E-commerce Product Excel Processor

Complete Learning Guide + Implementation Plan

What you're reading: a single document that (1) teaches the concepts behind this system in plain language, and (2) gives you the full, step-by-step plan to build it. Read it top to bottom the first time; use the table of contents as a reference later.

Table of Contents

1. The Big Picture (what we're building)
 2. Why S3 *and* a Database (a key confusion cleared up)
 3. Core Concepts — explained simply - 3.1 What is a Queue? - 3.2 What is RabbitMQ? - 3.3 What is BullMQ? - 3.4 RabbitMQ vs BullMQ (and why we use both) - 3.5 What is Event-Driven Architecture? - 3.6 Why every database query has a cost - 3.7 Why DELETE has the highest cost - 3.8 What is Soft Delete & why we (almost) always use it
 4. How THIS system is event-driven (full flow)
 5. The Tech Stack
 6. Required Excel Format
 7. Step-by-Step Implementation Plan
 8. How We'll Test It (verification)
 9. Running It Locally (this project)
 10. Local vs Production + Deployment & Costs
 11. Glossary (quick reference)
-

1. The Big Picture

You are building a **processor** — an automated pipeline that takes a messy Excel file full of products and turns the good rows into real products in your store, while telling the user exactly what went wrong with the bad rows.

In one sentence:

A client uploads an Excel sheet → the file is stored in AWS S3 → a background pipeline reads it row by row → valid products are created in the database → an Excel of only the *failed* rows is

generated → the user is emailed a summary, the required format, and that error file.

Why it matters: imagine a sheet with 10,000 products. You cannot make the user sit and wait while every row is checked and saved. So the work is done **in the background**, and the user is **notified by email** when it's finished. This "accept now, work later, notify when done" pattern is the whole point.

UPLOAD → STORE → PROCESS → NOTIFY
(instant) (S3) (background) (email)

2. Why S3 and a Database

This is the most important idea to get right. They sound similar but do **completely different jobs**.

	AWS S3 (object storage)	PostgreSQL (database)
What it holds	The raw .xlsx file , as one blob	Each product as a structured row
Can you ask "all red products under \$50"?	❌ No — it's just a file	✅ Yes — that's what a DB is for
Cost	Very cheap per GB	More expensive, but queryable
Analogy	A filing cabinet holding a sealed envelope	A spreadsheet where every line is searchable

The flow that ties them together:

```
Client uploads Excel
|
▼
The FILE goes to S3          ← storage (cheap, durable)
|
▼
Processor opens the file, reads each line
|
▼
Each LINE becomes a PRODUCT ROW in PostgreSQL ← usable data
```

Takeaway: S3 keeps the *original upload* safe and cheap. The database makes the *products* usable by the rest of your store (search, cart, inventory). You need both.

💡 **"Are we really using S3?" — Yes.** The code uses the **AWS S3 SDK**. During *local development* we point that same code at **MinIO** — free, open-source software that speaks the identical S3 API — so you can build and test for **\$0** without an AWS account. Going to production is a one-line `.env` change (remove the local endpoint) with **no code changes**. Details in §10.

3. Core Concepts

3.1 What is a Queue?

A **queue** is a *waiting line for work*. One part of the system drops a task into the line and walks away instantly. Another part (a "worker") picks tasks off the line and does them later.

```
Producer —put→ [ task | task | task ] —take→ Worker  
                (the queue)
```

This is what lets the upload return **instantly** — it just drops a task in the line instead of doing the slow work itself.

3.2 What is RabbitMQ?

RabbitMQ is a **message broker** — a system that carries **messages** from one service to another.

- A **producer** sends a message ("a file was uploaded!").
- RabbitMQ **routes** it.
- A **consumer** picks it up whenever it's ready.

📮 **Analogy — the post office.** You drop a letter in the mailbox. You don't know or care who delivers it or when the recipient reads it. The post office handles routing. That decoupling is RabbitMQ's superpower.

Best at: communication *between* services and broadcasting events.

3.3 What is BullMQ?

BullMQ is a **job queue** for Node.js, built on top of **Redis**. It is specialized for running **background jobs reliably**.

It gives you, for free: -  **Retries** — if a job fails, try again automatically. -  **Concurrency** — run many jobs at once. -  **Delays & scheduling** — "run this in 5 minutes." -  **Progress tracking** — "job is 60% done." -  **A dashboard** — see jobs succeed/fail.

 **Analogy — a smart to-do list for workers.** Each task is written down. If a worker drops one, it goes back on the list to retry. Nothing is silently lost.

Best at: *doing the actual heavy work* (like processing 10,000 rows).

3.4 RabbitMQ vs BullMQ — and why we use BOTH

They overlap a little, but their *jobs* are different:

	RabbitMQ	BullMQ
Category	Message broker / event bus	Job queue / task runner
Its job	"Tell everyone something happened"	"Actually do the heavy task, reliably"
Strength	Decoupling, routing, pub/sub	Retries, concurrency, scheduling, progress
In our app	Announces file uploaded	Runs the Excel-processing job

 **Simple mental model:** - **RabbitMQ** = the **doorbell** ringing — "*a package arrived.*" - **BullMQ** = your **to-do list** — "*unpack it, sort it, file it, retry if you drop something.*"

Why both here? It's a deliberate learning choice. It shows the clean separation between *messaging* (an event happened) and *job execution* (do the reliable work). A minimal production version could use BullMQ alone — but then you wouldn't see the distinction.

3.5 What is Event-Driven Architecture (EDA)?

An architecture where components talk by **emitting and reacting to events** instead of calling each other directly.

Direct (non-event) style:

```
Upload() → validate() → save() → makeErrorFile() → sendEmail()  
(one long function; the user waits for ALL of it)
```

Event-driven style:

```
Upload() —emits→ "FileUploaded" → (worker reacts) processes
                                   —emits→ "Processed" → (mailer reacts) emails
(each step is independent; the user waits for none of it)
```

Benefits: -  **Loose coupling** — parts don't depend on each other's internals. -  **Scalable** — add more workers to react faster. -  **Resilient** — if one part is down, events wait; nothing is lost. -  **Responsive** — the API answers instantly.

3.6 Why every database query has a cost

A query is never "free." Each one spends real resources:

Resource	What it's spent on
CPU	Parsing SQL, planning the best path, executing it
Disk I/O	Reading data pages from disk
Memory	Buffers, sorting, temporary results
Locks	Coordinating with other queries (concurrency)
Network	Sending results back

The more rows the DB must scan, the more it costs. **Indexes** make *reads* cheaper (jump straight to the row) — but they make *writes* more expensive, because every insert/update/delete must also update every index.

3.7 Why DELETE has the highest cost

A **DELETE** is far more than "erase a row." The database must:

1. **Find** the target rows (a read/scan first).
2. **Lock** them so nobody else touches them mid-delete.
3. **Remove them from every index** on the table — not just the table itself.
4. **Write to the transaction log (WAL)** so it can be rolled back / survive a crash.
5. **Handle foreign keys, cascades, and triggers** (which may delete *more* rows).
6. **Leave dead rows behind** ("dead tuples") that later need cleanup (**VACUUM** in Postgres), causing **table bloat**.

DELETE one row → touches: the table + ALL indexes + the log
+ FK checks + future cleanup

That's why a single delete can be heavier than a read *or* an update. It ripples across the whole table's machinery.

3.8 What is Soft Delete — and why we (almost) always use it

Soft delete = don't physically remove the row. Instead, **mark it** as deleted with a `deletedAt` timestamp (or an `isDeleted` flag). Queries then simply ignore rows where `deletedAt` is set.

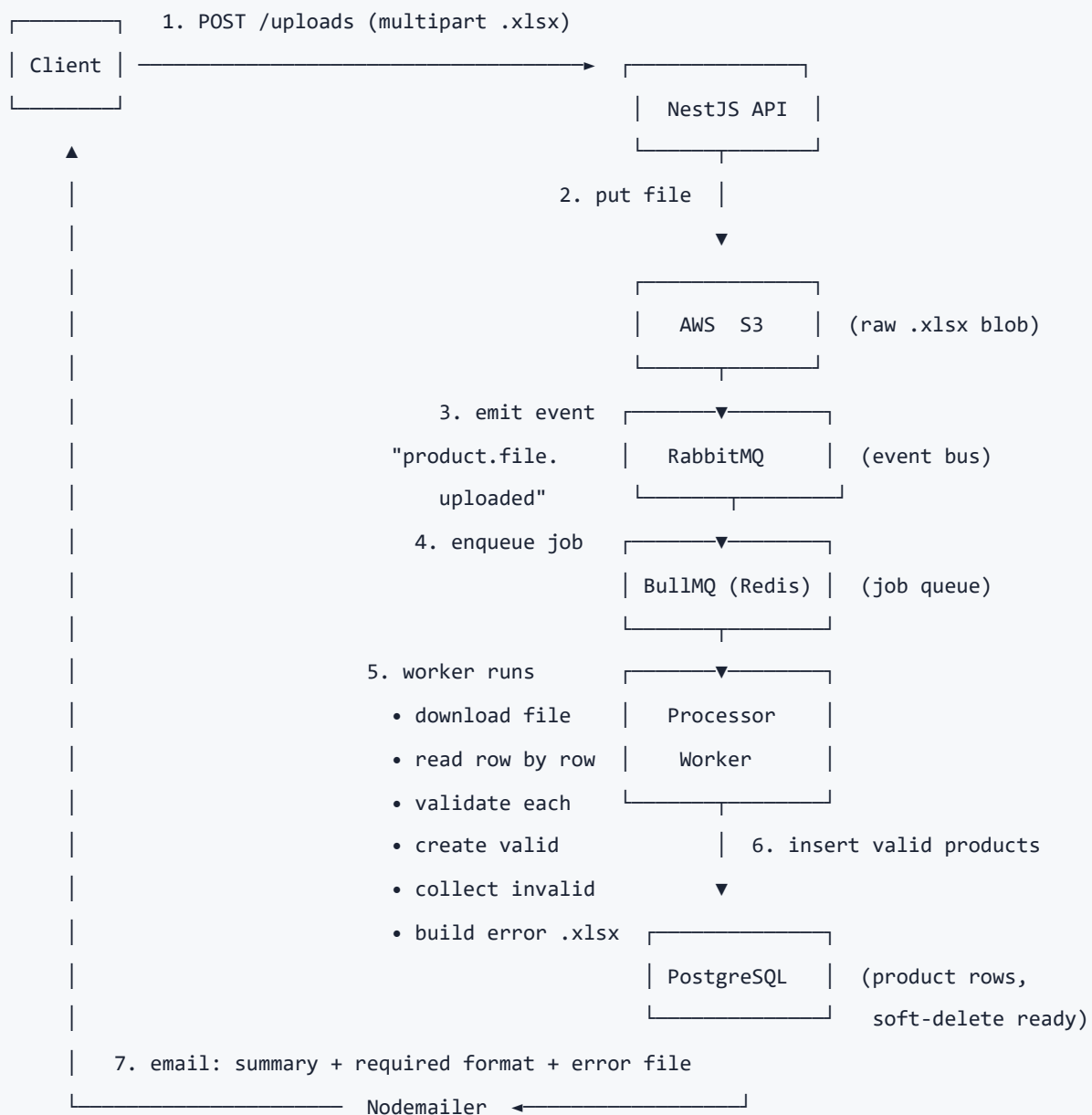
Hard delete: row is GONE forever. `DELETE FROM products WHERE id = 5;`
Soft delete: row stays, marked hidden. `UPDATE products SET deletedAt = now() WHERE id = 5;`

Why we prefer it: - 🔄 **Recoverable** — undo a mistake by clearing the flag ("restore"). - 📖 **Audit & history** — you keep a record of what existed (compliance, analytics). - 🔗 **Referential safety** — old orders still point to the product without breaking. - 💰 **Cheaper at delete time** — it's a lightweight `UPDATE`, not the heavy physical delete of \$3.7.

Trade-off: the table grows over time, so you index/filter on `deletedAt` and occasionally archive very old rows.

4. How THIS System Is Event-Driven

Here's the entire pipeline, and the events that drive it:



The events, in order:

#	Event	Who emits it	Who reacts	Result
1	<code>product.file.uploaded</code>	Upload API (RabbitMQ)	Messaging consumer	Adds a BullMQ job
2	(BullMQ job) <code>process-file</code>	Messaging consumer	Processor worker	Reads, validates, saves
3	<code>product.file.processed</code>	Processor (RabbitMQ)	Notification consumer	Sends the email

This is why it's event-driven: no single function does everything. Each stage *emits* when it's done, and the next stage *reacts*. The user's upload request finishes at step 1.

5. The Tech Stack

Layer	Choice	Why
Framework	NestJS + TypeScript	Modular; first-class BullMQ + RabbitMQ support
File storage	AWS S3 (@aws-sdk/client-s3)	Cheap, durable storage for the raw upload
Message broker	RabbitMQ (@nestjs/microservices)	Event bus between stages
Job queue	BullMQ + Redis (@nestjs/bullmq)	Reliable background jobs
Database	PostgreSQL + Prisma	Structured products, soft delete, indexing
Excel read/write	ExcelJS	Read the upload, generate the error sheet
Validation	class-validator	Enforce required fields per row
Email	Nodemailer	Send summary + format + error file
Upload handling	Multer (FileInterceptor)	Receive the multipart upload
Config	@nestjs/config	Manage secrets via .env

6. Required Excel Format

Each **row** = **one product**. A header row is required. All columns are required unless noted.

Column	Type	Rule
sku	string / number	required, unique
name	string	required, non-empty
description	string	required
price	number	required, > 0
category	string	required
color	string	required
stock	integer	required, ≥ 0

- A row is **successful** when every required field passes validation.

- A row is **unsuccessful** when any field is missing or malformed.
- The **error Excel** returns each failed row *plus an* `errors` column explaining exactly what's wrong.
- The **email** states this required format so the user can fix and re-upload.

Example error sheet:

sku	name	price	...	errors
(blank)	Blue Mug	12	...	sku is required
1002	Red Vase	-5	...	price must be > 0
1003	(blank)	abc	...	name is required; price must be a number

7. Step-by-Step Implementation Plan

Each step lists **what we do**, **files touched**, **commands**, and the **outcome** to verify before moving on.

STEP 0 — Prerequisites

- Confirm Node 20+, Docker Desktop, an AWS S3 bucket, and an SMTP account (dev: Mailtrap/Ethereal).
- ☒ *Outcome:* `node -v` and `docker -v` work; you have S3 keys + SMTP creds.

STEP 1 — Project scaffold

```
npm i -g @nestjs/cli
nest new ecommerce-processor
npm i @nestjs/config @nestjs/bullmq bullmq ioredis @nestjs/microservices amqplib amqp-connection-manage
npm i @aws-sdk/client-s3 exceljs nodemailer class-validator class-transformer
npm i -D prisma && npm i @prisma/client
npx prisma init
```

- ☒ *Outcome:* `npm run start:dev` boots a blank Nest app on `:3000`.

STEP 2 — Local infrastructure (Docker)

- Write `docker-compose.yml` with **Postgres**, **Redis**, **RabbitMQ** (+ management UI).
- `docker-compose up -d`
- ☒ *Outcome:* Postgres `:5432`, Redis `:6379`, RabbitMQ `:5672` + UI `:15672` healthy.

STEP 3 — Config + env wiring

- Load env via `@nestjs/config` ; typed config for S3, SMTP, DB, Redis, RabbitMQ.
- Files: `.env` , `.env.example` , `src/config/configuration.ts` , `src/app.module.ts` .
-  *Outcome:* app reads all secrets; a missing var throws on boot (fail-fast).

STEP 4 — Database schema (Prisma)

- Define `Product` (sku unique, name, description, price, category, color, stock, `deletedAt` , timestamps) and `UploadBatch` (fileKey, userEmail, counts, status enum `PENDING|PROCESSING|COMPLETED|FAILED`).
- `npx prisma migrate dev --name init`
-  *Outcome:* tables exist; `npx prisma studio` shows empty `Product` + `UploadBatch` .

STEP 5 — S3 service

- Implement `putObject` (upload buffer → return `fileKey`) and `getObject` (download).
- Files: `src/s3/s3.service.ts` , `src/s3/s3.module.ts` .
-  *Outcome:* can upload a dummy file and read it back from S3.

STEP 6 — Upload endpoint (producer)

- `POST /uploads` with `FileInterceptor` (`.xlsx` only, size limit). Flow: validate → `s3.putObject` → create `UploadBatch` (`PENDING`) → `emit` `product.file.uploaded {batchId, fileKey, userEmail}` → return **202** with `batchId` .
- Files: `src/upload/upload.controller.ts` , `upload.service.ts` , `upload.module.ts` .
-  *Outcome:* posting a file returns **202** instantly; file in S3; `PENDING` batch row; message in RabbitMQ UI.

STEP 7 — RabbitMQ messaging layer

- Configure Nest RabbitMQ microservice (`main.ts` `connectMicroservice`). Consumer `@EventPattern('product.file.uploaded')` **adds a BullMQ job** to `product-processing` .
- Files: `src/main.ts` , `src/messaging/messaging.module.ts` , `messaging.controller.ts` .
-  *Outcome:* emitting the event creates a BullMQ job in Redis.

STEP 8 — BullMQ worker: read + validate (core)

- Register queue `product-processing` ; implement `@Processor` . Inside the job: 1. set batch → `PROCESSING` ; 2. `s3.getObject` ; 3. **stream-read** rows with ExcelJS; 4. map each row → `ProductRowDto` + validate; 5. split into `valid[]` / `failed[] (+reasons)` .

- Files: `src/processing/processing.module.ts` , `processing.processor.ts` , `dto/product-row.dto.ts` , `excel.service.ts` .
-  *Outcome:* logs show `read=N`, `valid=X`, `invalid=Y` .

STEP 9 — BullMQ worker: persist products

- Insert `valid[]` via **chunked** `createMany` (e.g. 500/batch), `skipDuplicates` on `sku` . Update batch counts.
- Files: `processing.processor.ts` , `src/product/product.service.ts` .
-  *Outcome:* valid products in Postgres; batch counts correct.

STEP 10 — BullMQ worker: build error Excel

- With ExcelJS, write a new workbook: original columns + `errors` column; upload to S3 as `errors/{batchId}.xlsx` .
- Files: `src/processing/excel.service.ts` .
-  *Outcome:* an error `.xlsx` with only failed rows + reasons.

STEP 11 — Completion event + reliability

- On success: batch → `COMPLETED` , **emit** `product.file.processed {batchId, counts, errorFileKey, userEmail}` . Configure queue **retries** (attempts + backoff) and **concurrency**.
-  *Outcome:* killing the worker mid-job → retry; on success the processed event fires.

STEP 12 — Email notification

- `@EventPattern('product.file.processed')` → build email: summary (read/success/failed) + **required format table** + **attach** the error `.xlsx` (or link if large). Send via Nodemailer.
- Files: `src/notification/notification.module.ts` , `mailer.service.ts` , `notification.controller.ts` .
-  *Outcome:* email lands in Mailtrap with correct numbers + attachment.

STEP 13 — Product endpoints + soft delete

- `GET /products` (excludes `deletedAt != null`), `GET /products/:id` , `DELETE /products/:id` → **soft delete**, `POST /products/:id/restore` . Prisma extension auto-filters soft-deleted rows.
- Files: `src/product/product.controller.ts` , `product.service.ts` .
-  *Outcome:* delete hides but keeps the row; restore brings it back.

STEP 14 — Hardening & DX

- Global validation pipe, exception filters, request logging, `/health`, optional **Bull Board** dashboard at `/queues`, sample fixtures (`samples/valid.xlsx`, `samples/mixed.xlsx`), `README.md`.
-  *Outcome:* clear errors, a queue dashboard, ready-to-use samples.

STEP 15 — Deliverable docs

- This guide as `docs/PROCESSOR_GUIDE.md` **and** `docs/PROCESSOR_GUIDE.pdf` (identical content).
-  *Outcome:* both files exist with matching, structured content.

Target file structure:

```
ecommerce-processor/  
├─ docker-compose.yml  
├─ .env(.example)  
├─ prisma/schema.prisma  
├─ src/  
│   ├─ main.ts                # HTTP + RabbitMQ microservice bootstrap  
│   ├─ app.module.ts  
│   ├─ config/  
│   ├─ s3/s3.service.ts       # S3 put/get  
│   ├─ upload/upload.controller.ts # POST /uploads  
│   ├─ messaging/             # RabbitMQ event patterns  
│   ├─ processing/processing.processor.ts # BullMQ worker (core logic)  
│   ├─ processing/dto/product-row.dto.ts # validation rules  
│   ├─ processing/excel.service.ts # ExcelJS read + error-sheet write  
│   ├─ notification/mailer.service.ts # Nodemailer  
│   └─ product/product.service.ts # soft delete + queries  
└─ docs/PROCESSOR_GUIDE.md / .pdf
```

8. How We'll Test It

1. `docker-compose up -d` → Postgres, Redis, RabbitMQ up; `npx prisma migrate dev`.
2. `npm run start:dev` → confirm it connects to all three + S3.
3. **Happy path:** upload a mixed `.xlsx` (valid + invalid) via `POST /uploads`. Expect **202** immediately.
4. **Watch the flow:** logs show RabbitMQ event → BullMQ job → rows processed.
5. **DB check:** valid products inserted; `UploadBatch` counts correct.

6. **Email check** (Mailtrap/Ethereal): summary correct, required format shown, **error** `.xlsx` **attached** with only bad rows + `errors` column.
7. **Soft delete**: `DELETE /products/:id` → row stays with `deletedAt` ; `GET /products` hides it; `restore` brings it back.
8. **Resilience**: kill the worker mid-job → BullMQ retries; file still safe in S3.
9. **Docs**: confirm `.md` and `.pdf` exist with identical content.

9. Running It Locally

The project is built and **runs entirely on your machine for free** (Docker + MinIO + Ethereal). No cloud accounts needed.

Requirements: Node.js 20+, Docker Desktop.

```
# 1. Install dependencies
npm install

# 2. Create your env file (local defaults already work)
cp .env.example .env          # Windows CMD: copy .env.example .env

# 3. Start infrastructure: Postgres, Redis, RabbitMQ, MinIO
docker compose up -d

# 4. Create the database tables
npx prisma migrate dev --name init

# 5. Generate sample Excel files (samples/valid.xlsx, samples/mixed.xlsx)
npm run seed:sample

# 6. Run the app (HTTP API + background worker + event listener)
npm run start:dev
```

Upload a file and watch it flow:

```
curl -X POST http://localhost:3000/uploads \
  -F "file=@samples/mixed.xlsx" \
  -F "email=you@example.com"
```

You get `202 Accepted` instantly. The app logs then show: RabbitMQ event → BullMQ job → rows processed → **an Ethereum email preview URL** (open it to see the summary + attached error sheet).

Local dashboards:

Tool	URL	Login
RabbitMQ management	http://localhost:15672	guest / guest
MinIO console	http://localhost:9001	minioadmin / minioadmin
Prisma Studio (DB)	run <code>npx prisma studio</code>	—
Email previews	printed in the app logs (Ethereal)	—

10. Deployment and Costs

The golden rule

Local dev uses **MinIO** (S3-compatible) and **Ethereal** (test email). Production swaps in **real AWS S3** and a **real SMTP** provider — **by editing `.env` only, no code changes.**

Concern	Local dev	Production
Object storage	MinIO (Docker, free)	AWS S3
Email	Ethereal (free)	AWS SES / Mailtrap / SendGrid
Switch	<code>S3_ENDPOINT=http://localhost:9000</code>	remove <code>S3_ENDPOINT</code> → real AWS

What production needs

Component	Managed option
App (API + worker)	Docker container on Render / Railway / Fly.io / AWS ECS / VPS
PostgreSQL	Neon / Supabase / AWS RDS / Railway
Redis (BullMQ)	Upstash / AWS ElastiCache / Redis Cloud
RabbitMQ	CloudAMQP / AWS MQ
Object storage	AWS S3
Email	AWS SES / Mailtrap / SendGrid

Deployment steps (high level)

- 1. Provision: S3 bucket + IAM user (S3-only), managed Postgres, Redis, RabbitMQ, SMTP account.
- 2. Set the production `.env` (no `S3_ENDPOINT` ; real keys).
- 3. `npx prisma migrate deploy` against the prod DB.
- 4. Build the Docker image and deploy the container.
- 5. Scale the worker separately if volume grows.
- 6. Verify with the §8 checklist against production.

💰 Cost breakdown (free vs paid)

Free while developing locally (Docker on your PC): Postgres, Redis, RabbitMQ, MinIO, Ethereum — **\$0**.

Paid in production (approximate low-volume starting costs):

Service	Free tier?	Rough paid cost
AWS S3	5 GB free (12 mo)	~\$0.023/GB/mo + tiny request fees
Managed Postgres	Yes (Neon/Supabase)	~\$5–15/mo beyond free
Managed Redis	Yes (Upstash)	~\$0–10/mo low volume
Managed RabbitMQ	Yes (CloudAMQP free plan)	~\$0–19/mo
Email (AWS SES)	3,000/mo free (12 mo)	~\$0.10 per 1,000 emails
App hosting	Yes (hobby tiers)	~\$5–7/mo per always-on service

Bottom line: build & demo everything for **\$0** on free tiers + local Docker. A small real deployment typically runs **~\$10–40/month** total. Any paid step will be flagged before you take it.

11. Glossary

Term	One-line meaning
Processor	The background pipeline that turns an Excel upload into products.
S3	Cheap object storage for the raw file (not queryable).
Database (PostgreSQL)	Structured, queryable store for product rows.
Queue	A waiting line for work so the API can respond instantly.
RabbitMQ	Message broker — announces that events happened.
BullMQ	Redis-based job queue — runs the heavy work with retries.
Event-Driven Architecture	Components emit/react to events instead of calling each other directly.
Query cost	The CPU/IO/memory/lock work every DB query consumes.
DELETE cost	Highest cost: touches table + all indexes + log + cleanup.
Soft delete	Mark a row <code>deletedAt</code> instead of physically removing it.
NestJS	The TypeScript framework we build on.
Prisma	The ORM (type-safe database access) we use.
ExcelJS	Library to read and write <code>.xlsx</code> files.
Nodemailer	Library to send emails.

End of guide — the `.pdf` version contains this exact content in a print-friendly layout.